and see *where* they're running by issuing the command:

```
$ docker stack ps myapp
ID   NAME                 IMAGE    …  DESIRED…  CURRENT…  …  …
92…  myapp_visualizer.1   docker…  …  Running   Running…
gm…  myapp_database.1     postgr…  …  Running   Running…
g8…  myapp_redis.1        redis:…  …  Running   Running…
xx…  myapp_web.1          robise…  …  Running   Running…
kh…  myapp_db-migrator…   robise…  …  Shutdown  Complet…
uz…  myapp_web.2          robise…  …  Running   Running…
hy…  myapp_web.3          robise…  …  Running   Running…
```

To scale a service down, you use the same command as before, but specify a lower number of replicas than are currently running. For example, to scale back to a single web container, we can run:

```
$ docker service scale myapp_web=1
```

Once again, in the visualizer, you should see only a single web container.

Although we can scale a service via the docker service scale command, sometimes we want to specify this at *deploy time*. In our docker-stack.yml file, we can specify the number of containers to run for a service with the replicas attribute under deploy.

Let's modify our docker-stack.yml to make our web service start two containers when it's deployed:

```
web:
  image: robisenberg/myapp_web:prod
  ports:
    - "80:3000"
  env_file:
    - .env/production/database
    - .env/production/web
  deploy:
    replicas: 2
```

Let's see this in practice. When we deploy the app:

```
$ docker stack deploy -c docker-stack.yml myapp
Updating service myapp_database (id: tjlxqvm3vxop9flor3u4q21y2)
Updating service myapp_visualizer (id: zame53zwvvmmm70m6njrf0zd2)
Updating service myapp_db-migrator (id: zy14mg5nosigzi9i0ysr2f51y)
Updating service myapp_web (id: vo89apdhl4y0sfobb47ly8ctp)
Updating service myapp_redis (id: sxum5csqxwcujgy0g7418fv9y)
```

we should now see our two web containers running in the visualizer.